LogSentinel

A command-line tool that analyzes web-server access logs (Apache/Nginx common log format) and detects suspicious activity such as brute-force logins, path scanners, known attack signatures and server-error spikes.

Real problem it solves: small companies often have logs but nobody reading them. LogSentinel turns a raw log into a short, prioritized list of attackers and incidents, and returns exit code `1` when something is found, so it can run in cron jobs or CI pipelines.

Requirements

Python 3.9 or newer. No third-party packages.

Project structure

Path	Purpose
`cli.py`	Command-line entry point (argparse with sub-commands, exit codes)
`logsentinel/models.py`	`LogEntry`, `Alert`, `ParseStats` dataclasses and custom exceptions
`logsentinel/parser.py`	Safe streaming parser: path validation, regex, size and line-length limits
`logsentinel/detectors.py`	Abstract `Detector` base class, four detectors, `@register` and `@timed` decorators, `analyze()`
`logsentinel/report.py`	Text, JSON and CSV output formatters
`sample_logs/access.log`	Demo log with injected attacks and deliberately malformed lines
`tests/test_logsentinel.py`	Unit tests, including malicious-input cases
`contributions/`	One markdown file per team member describing their work

Usage

```bash
python cli.py analyze --input sample_logs/access.log
python cli.py analyze -i sample_logs/access.log --format json --min-severity high
python cli.py analyze -i sample_logs/access.log -d bruteforce scanner -t 10 -w 120 -o report.csv -f csv
python cli.py list-detectors
```

Option	Meaning	Default
`-i, --input`	Log file to analyze (required)	

`-d, --detectors`	Detectors to run	all
`-t, --threshold`	Events needed to raise an alert	5
`-w, --window`	Sliding window in seconds	60
`-f, --format`	`text`, `json` or `csv`	text
`-m, --min-severity`	`low`, `medium` or `high`	low
`-o, --output`	Write the report to a file	stdout

Exit codes: `0` clean, `1` alerts found, `2` error (bad file, bad argument).

Detectors

- **bruteforce**: many `401/403` responses from one IP inside a sliding window (uses a `deque`).
- **scanner**: one IP requesting many distinct missing (`404`) paths.
- **suspicious**: regex signatures for path traversal, SQL injection, XSS and secret files (`.env`, `.git`).
- **errorspike**: minutes with many `5xx` server errors.

Design and Python techniques used

- **Generators**: the file is streamed line by line, so memory stays constant for huge logs.
- **Context managers**: `@contextmanager` guarantees the file handle is closed.
- **Decorators**: `@register` builds a plugin registry, `@timed` measures analysis time.
- **OOP**: abstract base class (`ABC`) with one subclass per detector; adding a detector needs one new class.
- **Dataclasses and type hints** throughout.
- **Custom exception hierarchy** turned into clean error messages and exit codes.
- **Regular expressions** for parsing and attack signatures.

Robustness

The tool does not crash on hostile input: missing files, directories, binary data, invalid UTF-8, files over 200 MB, lines over 4096 characters (drained without storing), fake IPs, invalid timestamps, negative or huge numeric arguments, and unwritable output paths. Bad lines are counted and skipped.

Running the tests

```bash
python -m unittest discover -s tests -t .
```